Aufgabenblatt 2

Übung zu Einführung in die Bildverarbeitung

Hanna Beitz, Malin Haas, Lasse Huber-Saffer, Tim Radke,
Teodora Rogozenco, Jannis Scheff, Christian Wilms

SoSe 2026

Ausgabe: 24. April 2026 - **Abgabe bis: 1. Mai 2026, 10:00 per Moodle!**

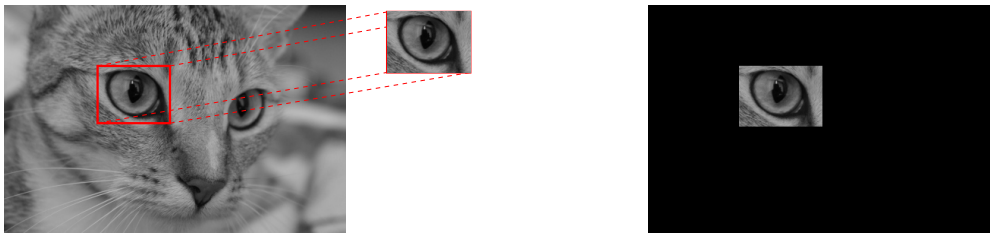
Lernziele

- Umgang mit Bildern in NumPy
- Abtastung, Quantisierung und Abtasttheorem verstehen
- Unterschiede zwischen Arten des Rauschens erfassen

Hinweis: Ohne nachvollziehbaren Rechenweg gibt es bei Theorieaufgaben keine Punkte!

Aufgabe 1 — Basisoperationen mit Bildern - 5+5+10=20 Punkte - Programmieraufgabe
Erlaubte (Sub-)Pakete: `numpy`, `matplotlib`, `skimage.io`
Diese Aufgabe dient dem ersten Kontakt mit Bildern in Python. Dazu sollt ihr ein gegebenes Bild manipulieren und alle Schritte in einem Python-Skript festhalten.

1. Zunächst soll ein Bild geladen, angezeigt und ein Ausschnitt gespeichert werden.
   a) Speichert euch das Bild `cat.png` aus dem Moodle ab und ladet es in Python mithilfe der Funktion `skimage.io.imread`.
   b) Zeigt das Bild mithilfe von Matplotlib an.
   c) Extrahiert den Bildausschnitt des rechten Auges der Katze als neues Array, wie in Abbildung 1a visualisiert. Zeigt den Bildausschnitt an.
   d) Speichert den Bildausschnitt mit der Funktion `skimage.io.imsave` ab.
2. Erzeugt nun ein Negativ des Originalbilds (`cat.png`). In einem Negativ sind alle Helligkeitswerte umgedreht. D.h. schwarz wird zu weiß, weiß wird zu schwarz und alle Werte dazwischen werden ebenso gedreht. Lasst euch das Bild zur Kontrolle anzeigen.
3. Erstellt nun eine Maske für den Bereich des rechten Auges der Katze.
   a) Erzeugt dazu zunächst ein neues Bild, das nur aus Nullen besteht, und die gleiche Größe wie das Originalbild hat. Dieses Bild wird eure Maske.
   b) Setzt nun in der Maske alle Pixel im Bereich des rechten Auges auf 1.
   c) Verrechnet anschließend Maske und Originalbild, sodass der Bereich außerhalb der Augenpartie schwarz ist und nur im Bereich der Augenpartie das eigentliche Bild zu sehen ist. Benutzt für diese Operation keine Schleife! Das Ergebnis sollte etwa wie in Abbildung **??** aussehen.



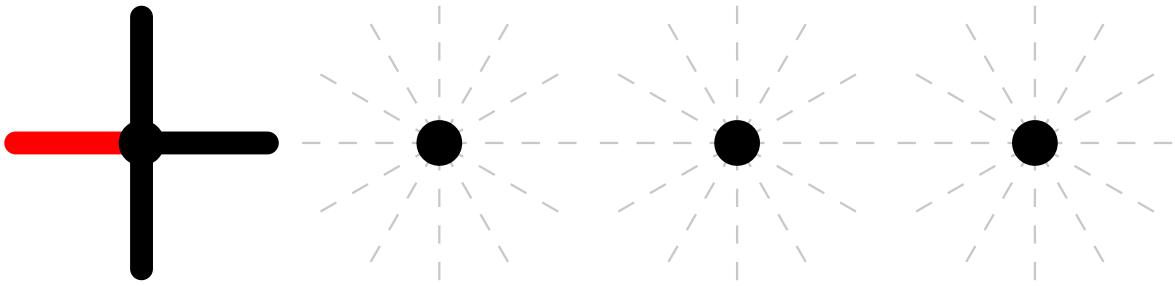
(a) (b)

Abbildung 1: **(a)**: Bildausschnitt des rechten Auges aus dem Originalbild (rechts) als neues Bild. **(b)**: Maskiertes Originalbild in dem nur das rechte Auge übrig ist.

Aufgabe 2 — Abtasttheorem - 15+10=25 Punkte - Theorieaufgabe
Ihr sitzt an Bord eines Propellerflugzeugs am Fenster mit bester Sicht auf einen der Propeller und könnt dessen Rotation klar und korrekt wahrnehmen. Nun wollt ihr die Rotation des Propellers mit eurer Kamera in einem Video festhalten. Ist dies aus Sicht der Signalverarbeitung eine sinnvolle Idee? Beantwortet dazu folgende Fragen schriftlich mit kurzer Begründung. Nehmt dazu an, dass sich der Propeller pro Minute 1200 mal um die eigene Achse dreht und eure Kamera eine Bildrate von 40 Hz hat, d.h., alle $\frac{1}{40}$s ein vollständiges Bild aufnimmt. Nehmt weiter an, dass eure Belichtungszeit infinitesimal klein ist.

1. Ist das Abtasttheorem im Sinne der zeitlichen Abtastung der Propellerdrehung durch die Kamera erfüllt? Rechnet es nach!

2. Skizziert nun grob die Propellerstellung des zweiten, dritten und vierten Einzelbildes des Videos. Geht dabei davon aus, dass der Propeller zum Zeitpunkt des ersten Einzelbildes so positioniert ist, wie in Abb. 2a dargestellt und genau einen rotes Propellerblatt hat. Geht zudem davon aus, dass der Propeller sich im Uhrzeigersinn dreht. In welche Richtung scheint der Propeller basierend auf diesen Einzelbildern zu rotieren?



(a) erstes Einzelbild (b) zweites Einzelbild (c) drittes Einzelbild (d) viertes Einzelbild

Abbildung 2: Ausgangsposition des Propellers in den ersten vier Einzelbildern des Videos.

Aufgabe 3 — Abtastung und Aliasing - 15+5+10=30 Punkte - Programmieraufgabe
Erlaubte (Sub-)Pakete: `numpy`, `matplotlib`
Im Rahmen dieser Aufgabe sollt ihr eine sin-Funktion abtasten und so künstliche Bilder erzeugen. Es sind dabei **keine Schleifen** etc. erlaubt. Nutzt stattdessen Numpy-Operationen und Broadcasting.

1. Erzeugt ein Bild der Größe 100×100 Pixel, das nur mit Nullen gefüllt ist und den Elementtyp `float` besitzt. Berechnet nun für jede y-Koordinate des Bildes (0 bis 99) den Wert der Funktion $\sin(0.1 \cdot y)$. Für die y-Koordinate 0 also etwa $\sin(0.1 \cdot 0) = 1$. Setzt die entstehenden 100 Werte in die erste Bildzeile ein. Kopiert diese Bildzeile dann, sodass alle 100 Bildzeilen identisch aussehen. Hinweis: Wir nehmen an, dass die Funktion sin() Werte in Radians erwartet, wie etwa die Funktion `np.sin()`.
2. Normiert nun die Pixelwerte auf den Bereich 0 bis 255 anhand des Wertebereichs der sin-Funktion. Aus dem Wert -1 wird folglich 0, aus dem Wert 1 wird 255, und entsprechendes gilt für alle Werte dazwischen. Quantisiert nun die Pixelwerte in den bekannten Wertebereich $(0, 1, 2, \ldots, 255)$. Die Art der Rundung ist unerheblich.
3. Ändert nun die Frequenz der sin-Funktion so, dass Aliasing entsteht. Woran erkennt ihr dieses?

Aufgabe 4 — Salt-and-pepper Noise - 25 Punkte - Theorieaufgabe
Nehmt an, ihr habt drei Varianten eines Bildes bei dem ursprünglich jedes Pixel den Wert 42 hatte. Die drei Varianten sind jeweils mit einem individuellen salt-and-pepper noise (Wahrscheinlichkeit 1%) verrauscht. D.h. die Auftrittswahrscheinlichkeiten von schwarz und weiß sind jeweils 0.5%. Nun vergleicht ihr die Pixel der drei Varianten an der jeweils selben Position (bspw. oben links) und bildet den Median dieser drei Varianten. Wie groß ist nun die Wahrscheinlichkeit, dass dieser Median dem eigentlichen, unverrauschten Wert (42) entspricht?

Zusatzaufgabe 5 — Rauschen - 10+10+5=25 Punkte - Programmieraufgabe
Erlaubte (Sub-)Pakete: `numpy`, `skimage.io`, `matplotlib`
In dieser Aufgaben sollen Bilder künstlich mit Rauschen gestört werden.

1. Erstellt eine Python-Funktion, die das Gaußsche Rauschen auf ein Bild applizieren. Dabei soll die Standardabweichung als Parameter anzugeben sein. Achtet darauf, dass das Ergebnis wieder im Wertebereich $\{0, 1, 2, \ldots, 255\}$ sein muss. Testet die Funktion am Bild `mandrill.png`.
   Hinweis 1: Nutzt das Paket `numpy.random`, um zufällige Werte zu ziehen.
   Hinweis 2: Mit der Funktion `numpy.clip` lässt sich ein bestimmter Wertebereich für ein Array erzwingen.
2. Schreibt eine zweite Funktion, die das salt-and-pepper noise auf ein Bild appliziert. Die Funktion soll die Wahrscheinlichkeit für eine Veränderung (salt und pepper sind dann gleich wahrscheinlich) als Parameter erhalten. Testet auch diese Funktion am Bild `mandrill.png`.
3. Welche Auswirkungen haben die Veränderungen der jeweiligen Parameter der beiden Funktionen (Standardabweichung bzw. Wahrscheinlichkeit für eine Veränderung) auf das Ergebnis?